Assignment 2, Part 3(a): CUBIC vs. Reno vs. BBR

Ayusha Patra Kabileswaran Krishna Kumar Vihaan Kodeboyina Millan Kumar

Summary

We repeated the Question 1 and 2 measurements with CUBIC, Reno, and BBR on five public iperf3 servers, three times each (44 of 45 tests completed). **BBR had the highest goodput on every destination in every repetition** (78.1 Mbit/s mean vs. 52.1 for CUBIC and 43.6 for Reno), but it had 12–19 ms more queueing delay than CUBIC and Reno on every path and retransmitted about 11% of its bytes. CUBIC beat Reno in 13 of 14 matched comparisons, and the gap grew with path RTT. Our paths were in a *loss-limited* regime: CUBIC and Reno kept a roughly constant window of 530–610 segments on paths from 48 to 130 ms, so their goodput fell roughly as 1/RTT, while BBR, which does not treat loss as a congestion signal, was not limited this way.

Experimental setup

Host	Ubuntu 24.04.5 LTS VM (Multipass) on an Apple Silicon Mac, Wi-Fi uplink
Kernel	Linux `6.8.0-147-generic`, aarch64; mainline `tcp_bbr` (BBRv1) loaded with `sudo modprobe tcp_bbr`
Available algorithms	`reno cubic bbr` (`/proc/sys/net/ipv4/tcp_available_congestion_control`)
Software	Python 3.12.3, matplotlib 3.11.2
Command	`python3 main.py -n 5 -d 60 -i 0.2 -r 3 --seed 42`
Per test	60 s transfer, TCP_INFO sampled every 0.2 s, 128 KiB send blocks
Repetitions	3 per (destination, algorithm) pair, 45 tests planned

Destinations. Part 3 reuses the five destinations Part 1 reached (Part 1 needed 8 attempts: one server was busy, one timed out, and one refused the connection). Listed by minimum RTT:

Destination	Provider (country)	min RTT (ms)
`speedtest.mia11.us.leaseweb.net:5202`	LeaseWeb (US)	47.8
`148.230.45.213:30009`	FortiSASE (ES)	104.5
`speedtest.milkywan.fr:9228`	MilkyWan (FR)	109.3
`speedsrv.multinet24.pl:5307`	Multinet24 (PL)	130.1
`speedtestlondon.telecom.mu:5208`	Mauritius Telecom (MU)	288.1

Test order. In each repetition, every destination is tested with all three algorithms back to back, so the three algorithms on one path are measured within about 3.5 minutes of each other. The algorithm order rotates each repetition, so each algorithm runs first, second, and third once on every path:

Repetition	Order on each destination
1	CUBIC → Reno → BBR
2	Reno → BBR → CUBIC
3	BBR → CUBIC → Reno

Public iperf3 servers accept one client at a time and often refuse a new test for a few seconds after the previous one ends, so the runner pauses 3 s before each test and retries a refused test up to twice (after 10 s

and 30 s). Two tests were refused ("server busy"), both Reno on `speedtest.mia11.us.leaseweb.net`. The one in repetition 2 succeeded on its second retry. The one in **repetition 3 was refused all three times**, so that destination has two Reno runs instead of three. Every failed attempt is in `part3a-results/failures.csv`.

Algorithm selection. The runner sets the algorithm on the data socket with `setsockopt(TCP_CONGESTION)` before connecting, reads it back with `getsockopt`, and aborts the test if the kernel used a different one (`run_test.py` L150–170). The algorithm actually used is stored in every sample row.

Metrics

All metrics come from `TCP_INFO` on the data socket, as in Question 2. Per test we compute (`path_metrics`):

- **Goodput:** $8\,\Delta$`tcpi_bytes_acked`$/\Delta t$ per interval, summarized by min, median, mean, and p95.
- **RTT:** the kernel's smoothed RTT (`tcpi_rtt`); mean and p95 over the test.
- **min RTT:** `tcpi_min_rtt` at the end of the test, our estimate of the path's propagation delay.
- **Queueing delay:** mean RTT − min RTT.
- **Loss proxy:** retransmitted bytes as a percentage of bytes sent (`tcpi_bytes_retrans`/`tcpi_bytes_sent`), plus the per-interval retransmission count `delta_retrans` used in the plots.
- **BDP estimate:** median goodput × min RTT. This uses the goodput the algorithm *achieved*, so it is a lower bound on the path's true bandwidth-delay product, not a capacity measurement.

Results

Table 1: All tests per algorithm. Goodput spread here is dominated by differences between destinations; see Table 2 for run-to-run spread.

	Tests	Goodput (Mbit/s)	Mean RTT (ms)	Queueing delay (ms)	Retrans. bytes (%)	Mean cwnd (segments)
CUBIC	15	52.1	163.0	25.7	0.24	640
Reno	14	43.6	166.9	23.8	0.74	545
BBR	15	**78.1**	174.8	38.6	11.21	2538

Table 2: Per destination, mean ± standard deviation of per-run mean goodput across repetitions (3 runs, except Reno at LeaseWeb with 2).

Destination (min RTT)	Algo	Goodput (Mbit/s)	Mean RTT (ms)	Queueing delay (ms)	Retrans. bytes	Mean cwnd (segments)
LeaseWeb US (48 ms)	CUBIC	93.2 ± 1.4	77	28.7	0.04%	601
	Reno	85.7 ± 3.7	72	25.6	0.36%	538
	BBR	**144.5** ± 11.9	91	44.5	12.64%	2838
FortiSASE ES (105 ms)	CUBIC	48.6 ± 1.4	133	25.6	0.07%	588
	Reno	45.0 ± 1.6	129	23.3	1.01%	544
	BBR	**67.5** ± 1.4	141	35.7	9.08%	1961
MilkyWan FR (109 ms)	CUBIC	47.1 ± 2.0	132	24.1	0.05%	571
	Reno	45.0 ± 1.4	132	24.4	0.66%	546
	BBR	**72.8** ± 6.2	142	36.0	15.51%	2327
Multinet24 PL (130 ms)	CUBIC	41.5 ± 3.9	153	22.3	0.52%	605
	Reno	36.0 ± 2.1	155	23.8	0.54%	529
	BBR	**63.3** ± 5.3	168	37.5	11.43%	2360
Mauritius Tel. MU (288 ms)	CUBIC	30.0 ± 5.5	321	27.8	0.51%	834
	Reno	20.2 ± 2.5	316	22.4	1.00%	565
	BBR	**42.4** ± 1.9	332	39.3	7.39%	3207

Representative destination. The plots below use `148.230.45.213:30009`, the same representative destination as Question 2 (the Part 1 run closest to the median mean goodput). All three algorithms share the same axis scales on every plot.

The same time-series and scatter plots for each algorithm on its own page are in the appendix (Figures 4–6).

Analysis

What the traces show

Figure 1 shows each algorithm's characteristic window behavior. **Reno** grows its window linearly, by about one segment per RTT: from roughly 450 to 700 segments over 40 s at a 130 ms RTT, before a single loss halves it. **CUBIC** shows its cubic sawtooth: after each loss it drops to 70% of the window, grows back quickly, flattens out near the previous maximum, and then probes past it. It hits a loss every few seconds to about 15 s. **BBR**'s window is much larger (1,300–3,300 segments) and oscillates with its ProbeBW pacing cycle. It collapses almost to zero at about 12, 31, and 48 s; these are BBR's ProbeRTT phases, in which it drains the queue to re-measure the minimum RTT. Reno's large retransmission spike at about 2 s is the slow-start overshoot, which also accounts for most of Reno's retransmitted bytes on this path.

In the scatter plots (Figure 2), CUBIC and Reno samples cluster at windows of 300–800 segments and goodput of 25–75 Mbit/s, with almost no retransmissions. BBR's samples sit at larger windows and higher goodput, and BBR is the only algorithm with sustained retransmissions: hundreds of retransmitted segments per interval while still delivering 40–125 Mbit/s.

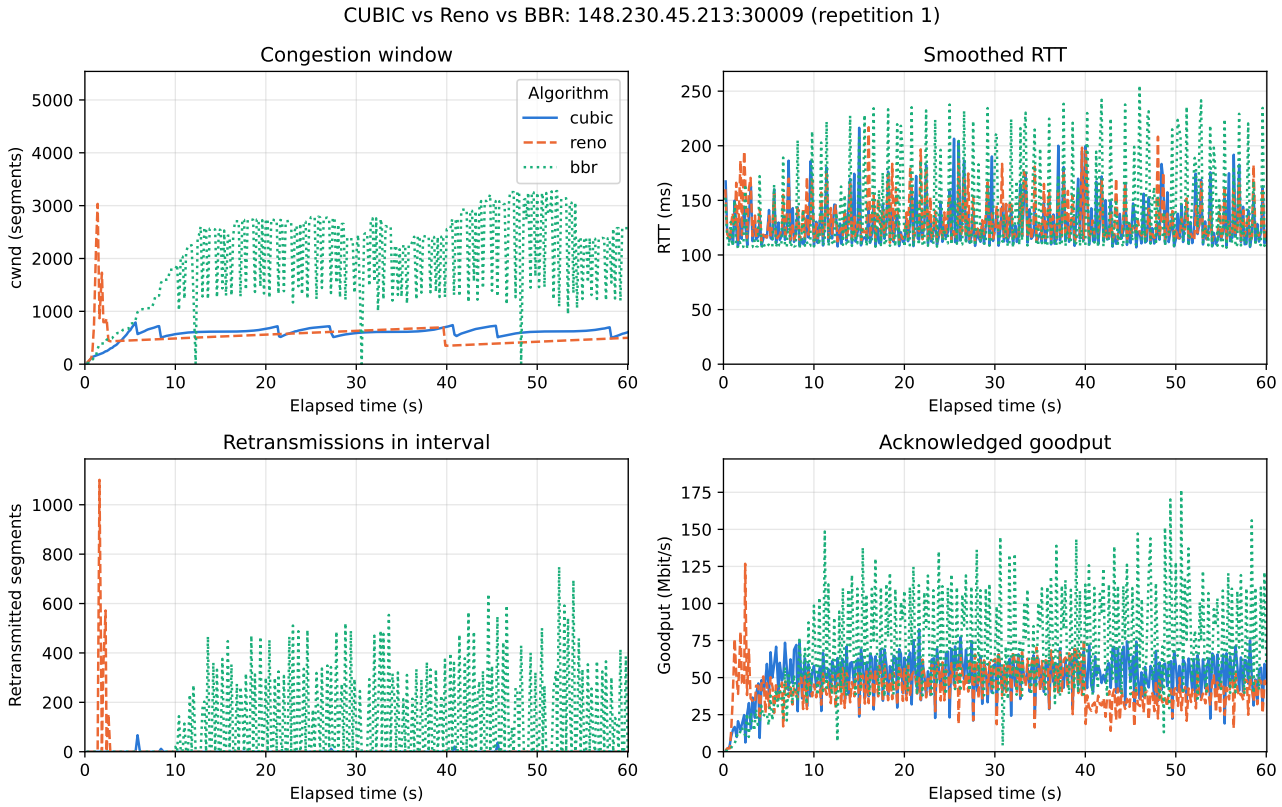


Figure 1: Time series at the representative destination, repetition 1: congestion window, smoothed RTT, retransmissions per 0.2 s interval (loss proxy), and acknowledged goodput. Line style and color identify the algorithm.

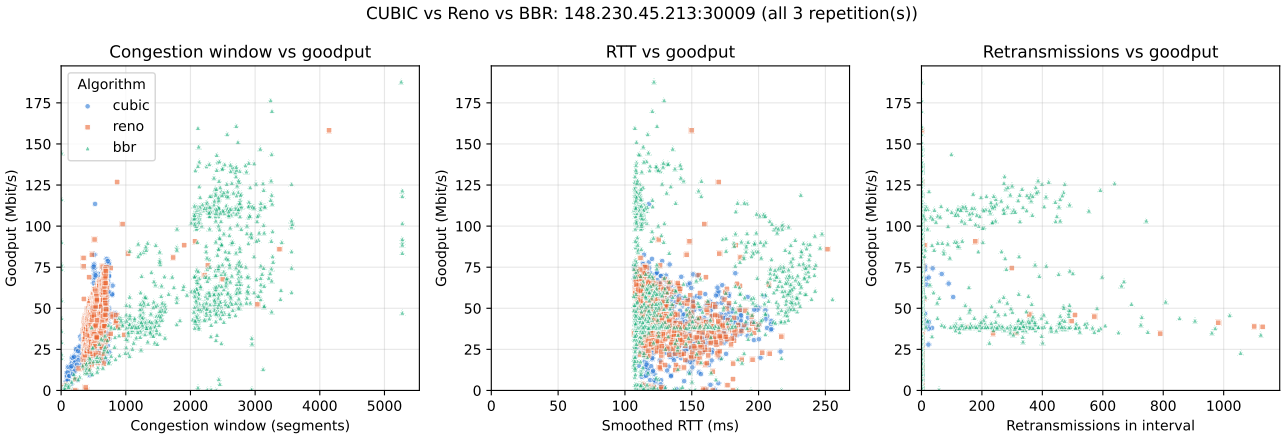


Figure 2: Congestion window, RTT, and retransmissions vs. goodput at the representative destination; every 0.2 s sample from all three repetitions.

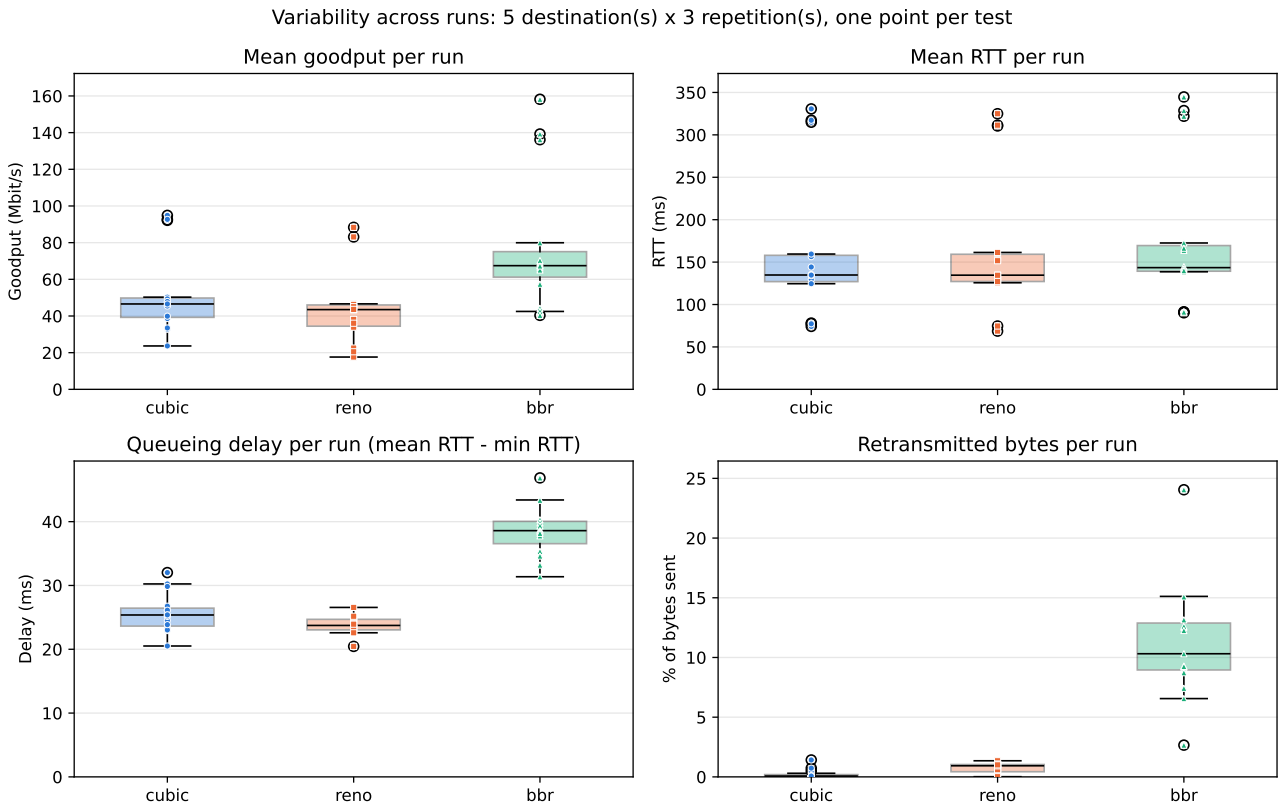


Figure 3: Run-to-run variability: one point per test across all destinations and repetitions.

The operating regime: loss-limited, not capacity-limited

The most important pattern is in Table 2. CUBIC and Reno kept nearly the same mean window, 530–610 segments (about 0.8 MB), on four paths whose minimum RTTs range from 48 to 130 ms. Since sending rate is roughly cwnd/RTT, their goodput fell roughly as 1/RTT, for example from 85.7 Mbit/s at 48 ms to 36.0 Mbit/s at 130 ms for Reno. If these flows were limited by bottleneck capacity, the window would instead grow to match each path's BDP.

A roughly constant window is what the classic AIMD throughput model predicts when the window is limited by the loss rate p rather than by capacity:

$$\text{throughput} \approx \frac{\text{MSS}}{\text{RTT}} \cdot \frac{1.22}{\sqrt{p}}, \qquad \text{i.e.} \qquad \text{cwnd} \approx \frac{1.22}{\sqrt{p}}.$$

Our Reno trace is consistent with this. One loss in about 40 s at about 45 Mbit/s is one loss per roughly 155,000 segments ($p \approx 6 \times 10^{-6}$), which gives cwnd $\approx 1.22/\sqrt{p} \approx 480$ segments, close to the 530–565 we measured. The losses are sparse: Reno and CUBIC retransmitted under about 1% of bytes, and most of that came from slow start. Each such loss still makes them cut the window, so they rarely grow large enough to fill the path.

BBR is not limited this way, because it does not use loss as its congestion signal. It estimates the bottleneck bandwidth and minimum RTT and paces at that rate, keeping up to about 2× the estimated BDP in flight (its window gain). This is why it reached 1.5–2.1× Reno's goodput on every path. It also shows that the paths had more capacity than CUBIC and Reno were using.

When each algorithm performs better

Throughput: BBR, increasingly so on long paths. BBR had the highest goodput on every destination in every repetition (15 of 15 comparisons). Its advantage over Reno grew with RTT, from 1.5–1.8× on the 48–130 ms paths to 2.1× on the 288 ms path. With a sparse loss rate, a loss-based algorithm needs a window close to the BDP to fill the pipe, and after each backoff it needs more round trips to regrow on a long path, whereas BBR's rate does not depend on loss. BBR is the better choice for bulk transfer on high-BDP paths with sparse non-congestive loss, the situation it was designed for.

CUBIC vs. Reno: CUBIC on high-BDP paths, about equal on short ones. CUBIC beat Reno in 13 of 14 matched comparisons. The gap was small on short paths (1.05–1.09× at 48–109 ms), 1.15× at 130 ms, and large on the longest one (1.48× at 288 ms, where CUBIC's mean window was 834 segments vs. 565 for Reno). Two design differences explain this. CUBIC backs off to 70% of its window instead of 50%, so it loses less after each loss. More importantly, its window grows as a function of *time since the last loss*, $W(t) = C(t-K)^3 + W_{\max}$, independent of RTT, whereas Reno adds one segment *per RTT*. On a 288 ms path, Reno needs over 2 minutes to regrow 500 segments, while CUBIC regrows in seconds. On short-RTT, small-BDP paths, Reno's per-RTT growth is fast enough that the two behave similarly.

Delay: Reno and CUBIC. Reno and CUBIC had similar queueing delay (23.8 and 25.7 ms mean; each was lower on some destinations), and BBR had the highest on every destination (38.6 ms mean). Part of this delay is common to all paths and algorithms: the 22–28 ms floor appears even for Reno on the shortest path, so it is probably jitter from our own access link (Wi-Fi) and the VM's virtual network rather than queueing on the Internet path. Above that floor, BBR added a consistent 12–19 ms of standing queue compared with the lower of CUBIC and Reno on the same path. BBRv1 keeps up to 2× the estimated BDP in flight and periodically paces at 1.25× its estimated bandwidth to probe for more, and that extra data waits in the bottleneck buffer. CUBIC and Reno back off as soon as the

buffer overflows, so they hold less standing queue. For latency-sensitive traffic on these paths, CUBIC or Reno is preferable.

Loss: BBR causes it, CUBIC avoids it. BBR retransmitted 7–16% of its bytes on every destination, compared with under about 1% for CUBIC and Reno. Because BBR does not slow down when packets are dropped, its probing overflows the bottleneck buffer repeatedly, and the dropped packets are resent. Its *goodput* is still highest because the retransmissions are excluded and it keeps the link busy, but this wastes capacity, and a BBR flow sharing a buffer with CUBIC or Reno flows would cause them losses and back them off. We did not measure competing flows, so this fairness cost is a prediction, not an observation.

Relating to the BDP. The achieved-goodput BDP lower bound grows from about 0.5–0.9 MB on the shorter paths to 0.8–1.5 MB on the 288 ms path (`bdp_kbytes` in `comparison_summary.csv`). CUBIC and Reno's mean windows of about 0.8 MB are near the BDP on the short paths but far below it on the long one, which is where they fall furthest behind BBR. BBR's windows of 2,000–3,200 segments (2.8–4.6 MB) are well above the BDP; the excess is what shows up as queueing delay and retransmissions.

Regimes we did not observe. With a deep, bloated buffer and no random loss, CUBIC and Reno would fill the buffer and BBR would be expected to have the *lower* RTT. On a short, uncongested path all three should reach link capacity and behave similarly. None of our five paths was in either regime, so we cannot confirm these predictions from our data.

Variability across runs

On a single destination, results were consistent across repetitions: the coefficient of variation of per-run mean goodput was 1.5–9.4% for 13 of the 15 destination–algorithm pairs (Table 2, Figure 3). The exceptions were CUBIC (18.3%) and Reno (12.2%) on the long Mauritius path, where a single loss costs the most and CUBIC's first run was much lower (23.7 vs. about 33 Mbit/s). By contrast, goodput across *different* destinations varied by a factor of 3–4 for every algorithm. The path matters more than the repetition, which is why we compare algorithms on the same path within minutes of each other.

The ranking was stable across runs and test orders: BBR was fastest in all 15 destination–repetition groups, and CUBIC beat Reno in 13 of 14 (the exception, MilkyWan in repetition 2, was a 0.9 Mbit/s difference). Because the order rotated, each algorithm ran first, second, and third once on every path, so a ranking caused by test order would have changed across repetitions; it did not.

Limitations of comparisons over the public Internet

- **Paths change.** Cross traffic, routing, and server load change between tests. Back-to-back testing and rotation reduce but do not remove this effect, and three repetitions per pair is a small sample.
- **One flow at a time.** We never ran algorithms against each other, so we cannot measure fairness or how BBR's losses would affect competing flows.
- **Unknown bottleneck.** We cannot see where the bottleneck is or how large its buffer is. Our host was a VM behind macOS NAT on Wi-Fi, which adds delay jitter and may be the bottleneck on some paths. The common 22–28 ms queueing-delay floor suggests this.
- **Servers.** Public iperf3 servers can rate-limit or refuse tests (one of our tests was refused three times), and their receive windows and load are unknown.
- **BDP is a lower bound.** We do not know the true bottleneck bandwidth, so our BDP estimate uses achieved goodput.

- **Measurement granularity.** TCP_INFO is sampled every 0.2 s and RTT is the kernel's smoothed estimate, so short bursts of loss or delay are averaged, and individual duplicate-ACK and timeout events are not separated.
- **One BBR version.** Linux 6.8 ships BBRv1; BBRv2/v3 react to loss and would likely retransmit much less.

Reproducing the results and code links

From `Assignment2/` on Linux with `reno`, `cubic`, and `bbr` available (Docker instructions are in the README):

```
sudo modprobe tcp_bbr
python3 -m pip install -r requirements.txt
python3 main.py -n 5 -d 60 -i 0.2 -r 3 --seed 42
```

Raw per-sample traces are written to `output/part3/{cubic,reno,bbr}/`, and summaries and plots to `output/`. The summaries, manifest, failure logs, and plots used in this report are copied in `Assignment2/part3a-results/`. The servers chosen depend on which public servers are reachable, so a re-run will not reproduce exactly the same destinations.

- Command-line options, including `-r/--repetitions`: `main.py` L45–96; Part 3 call: L123–137.
- Part 3 experiment loop (reuse of Part 1 destinations, rotating order, pause and retries): `run_part3`, repetition loop L279–323, `rotated_order`, `part1_destinations`.
- Per-run metrics and summaries: `path_metrics`, `_group_summary`.
- Algorithm selection and verification: `run_test.py` L150–170; goodput and loss-proxy computation: `compute_sample`.
- Plots (Figures 1–3 and the appendix): `plot_comparison`, `_overlay_time_series_page`, `_overlay_scatter_page`, `_variability_page`; representative-destination choice: `_part3_destination`.

Use of AI tools

We used Claude Code (Anthropic) to help set up the Linux VM, implement the Part 3 changes (destination reuse, repetitions and rotating order, retries, summary metrics, and comparison plots), and draft this writeup from our measurement files. We reviewed the code, tested it with short runs before the full experiment, and checked every number in this report against the CSV files in `part3a-results/`.

Appendix: per-algorithm plots

Repetition 1 at `148.230.45.213:30009`, with the same axis scales as Figures 1 and 2.

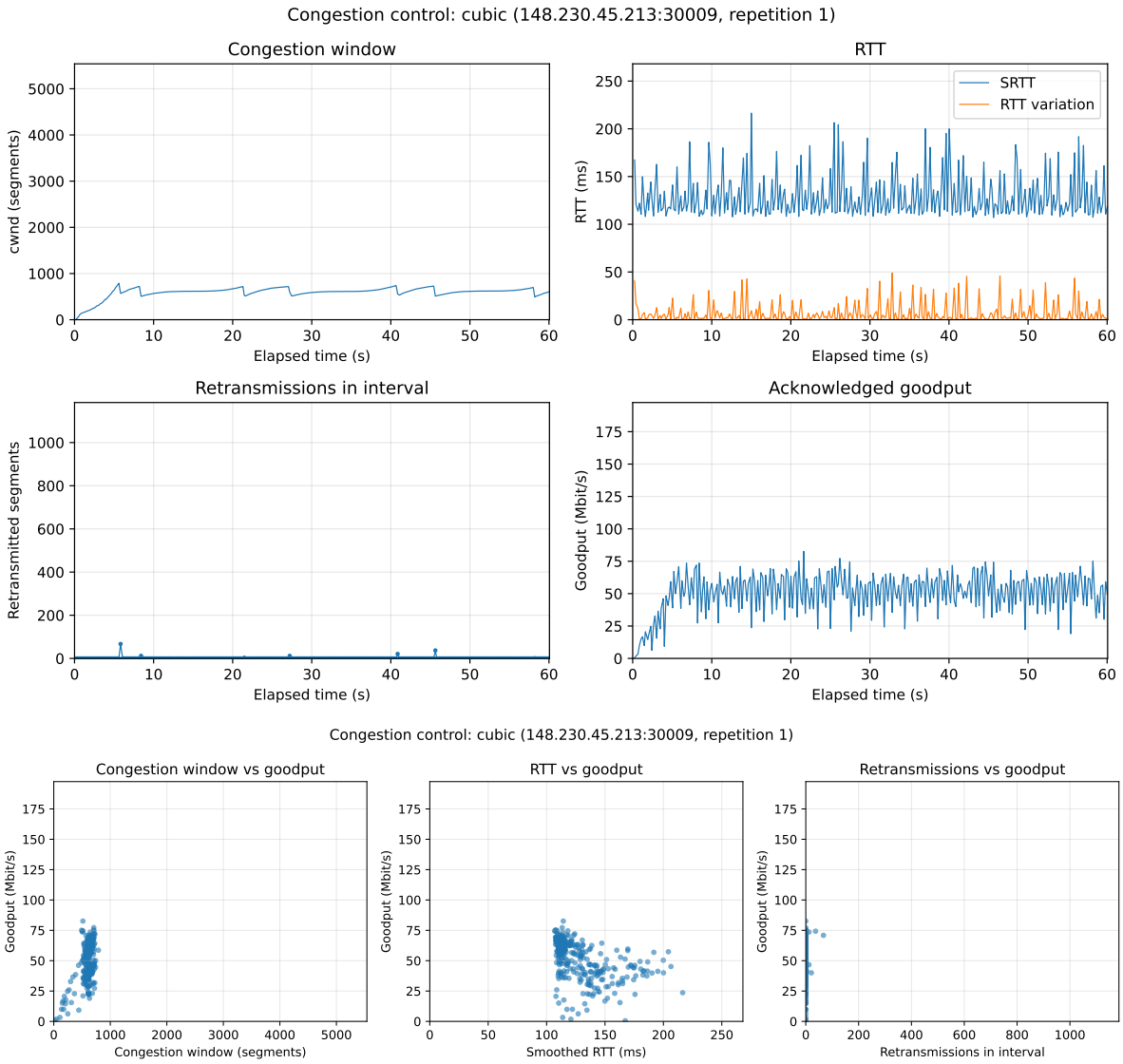


Figure 4: CUBIC: time series (top) and scatter plots (bottom).

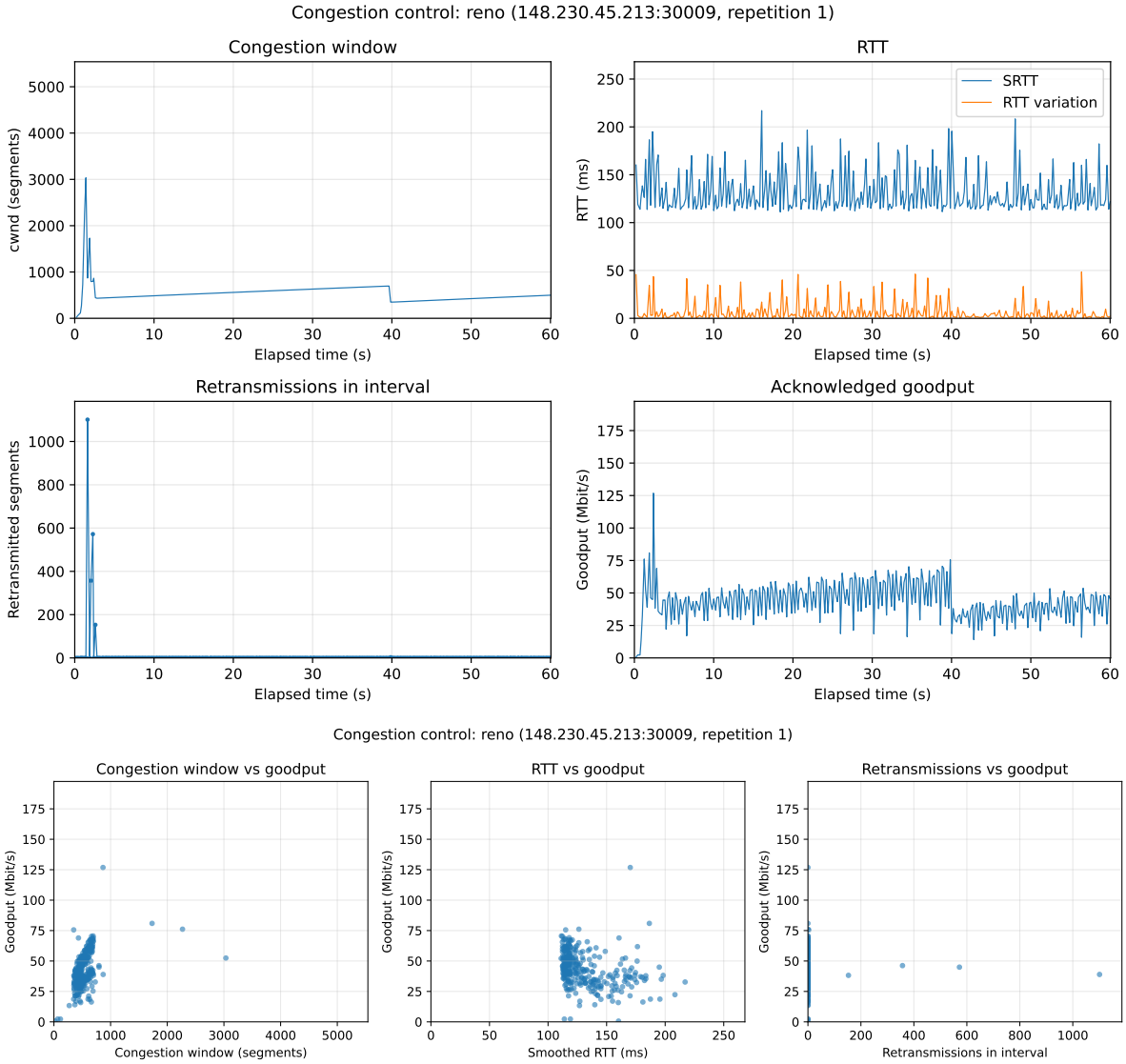


Figure 5: Reno: time series (top) and scatter plots (bottom).

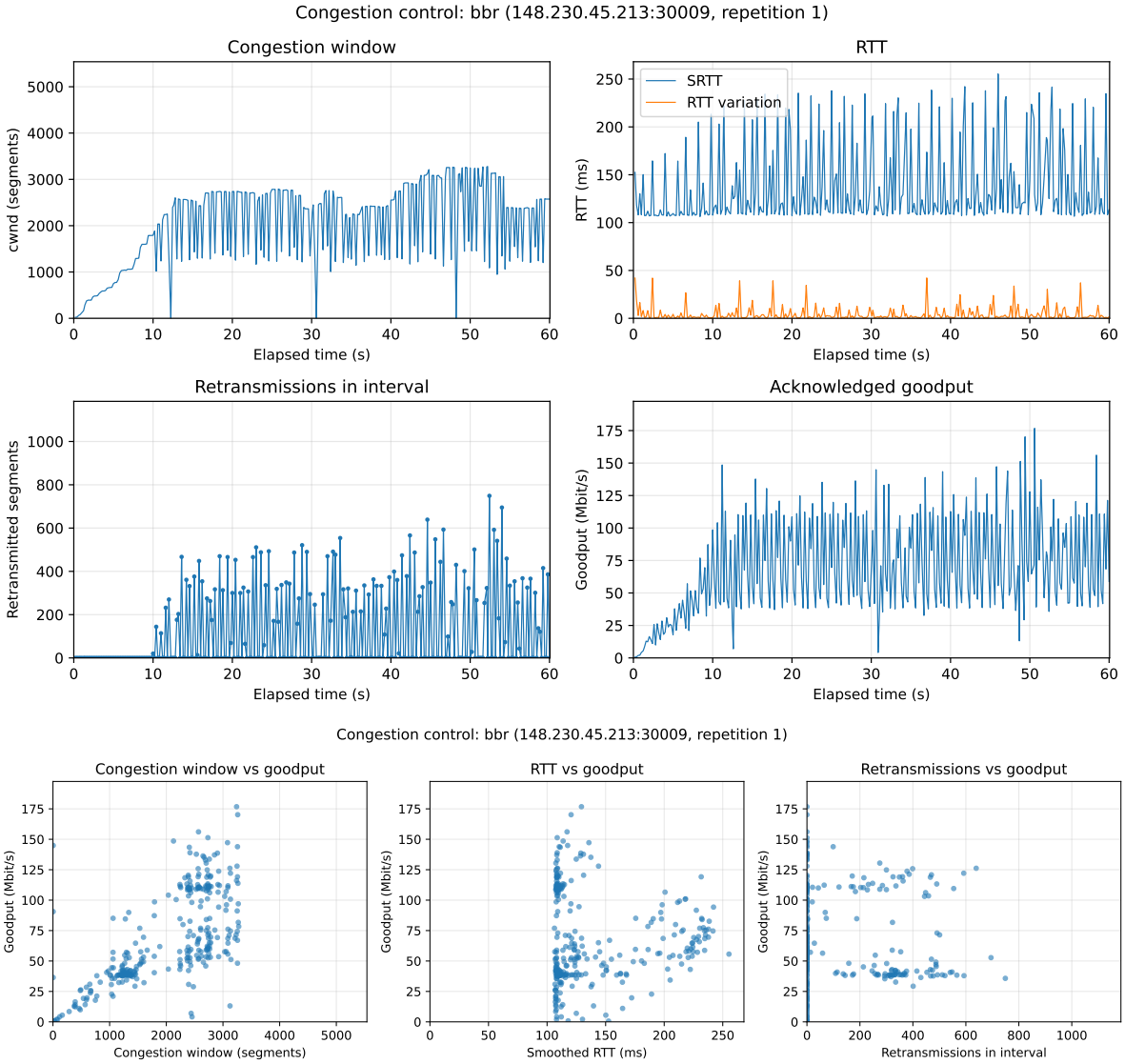


Figure 6: BBR: time series (top) and scatter plots (bottom).